

Laboratorium 8

Jako laboratorium do którego porównywane będą wyniki wybrałem laboratorium 6 dotyczące speed datingu (czyli działania na zbiorach nie zrównoważonych). Po przejrzaniu konkursów kaggle'a najbliższy mojemu zadaniu wydawał się konkurs Credit Card Fraud Detection (<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>), ponieważ też dotyczy on danych silnie niebalansowanych (0.17% fraudów) i jego cechy są znane (V1-V28 to wyniki PCA, plus Time i Amount). Jako rozwiązania, do których porównywałem swoje rozwiązanie przyjąłem następujące zgłoszenia:

<https://github.com/georgymh/ml-fraud-detection> (logistic-regression.ipynb, 280★)

<https://github.com/georgymh/ml-fraud-detection> (k-means.ipynb, 280★)

<https://github.com/yazanobeidi/fraud-detection> (project.ipynb, 108★)

EDA

W moim notatniku EDA jest dość rozbudowana — mam histogramy wieku, rozkład klas (bar + pie), match rate wg płci i rasy, histogramy samoocen z podziałem na match/brak, boxploty ważności cech, różnice hobby, analizę braków i heatmap korelacji. Mimo to brakuje kilku rzeczy które w notatnikach Kaggle robią różnicę.

W moim notatniku pojawia się na przykład wykres match rate vs samoocena atrakcyjności:

```
match_by_attr = df.groupby(attractive)[match].mean() * 100
ax.bar(match_by_attr.index.astype(int), match_by_attr.values, ...)
```

Pokazuje on że osoby oceniające siebie na 10 mają 22% match rate, a na 2 — 0%. Jest to jednak surowa średnia — nie bierze pod uwagę że osób oceniających się na 7 jest 2914 a na 2 tylko 20. Bardziej użyteczne byłoby pokazanie proporcji z oznaczeniem liczebności, tak jak robi to yazan w swoim notatniku gdzie dla każdej z 28 cech V1-V28 rysuje distplot z podziałem na fraud/normal:

```
fig = plt.figure(figsize=(12,28*4))
gs = gridspec.GridSpec(28,1)
for i, cn in enumerate(data.columns[:28]):
    ax = plt.subplot(gs[i])
    sns.distplot(data[cn][data.Class == 0], bins=50)
    sns.distplot(data[cn][data.Class == 1], bins=50)
    ax.set_xlabel()
    ax.set_title(histogram of feature:  + str(cn))
```

Dzięki temu widzi które cechy faktycznie różnicują klasy i usuwa te które tego nie robią (V28, V27, V23, V8). U mnie wszystkich 47 cech nie przeanalizowałem w ten sposób — zrobiłem to tylko dla 5 samoocen. Sprawdzenie wszystkich cech przez distploty mogłoby ujawnić które hobby lub preferencje faktycznie różnicują match/brak matchu.

Kolejna rzecz której nie robię a która pojawia się w notatniku yazana to szukanie anomalii na poziomie kilku kolumn jednocześnie. U mnie sprawdzam braki per kolumna, ale nie sprawdzam czy istnieją kombinacje wartości które nie mają sensu — np. osoba która deklaruje zainteresowanie jogą ale nie ćwiczeniami, albo kolumna `expected_num_matches` > 10 przy match rate = 0%. Yazan identyfikuje takie przypadki i usuwa je, co daje zauważalny wzrost wydajności.

Preprocessing

W moim notatniku preprocessing jest dość rozbudowany — usuwam 75 kolumn wycieku wiedzy, tworzę odcisk palca użytkownika przez dyskretyzację profilu i hash md5, dzielę dane per użytkownik (5 ostatnich interakcji do testu), dodaję 6 nowych cec.

W pliku yazana stosowana jest dogłębna analiza outlierów — sprawdza on czy dla danych niestandardowych proporcja klasy docelowej jest faktycznie inna. U mnie na przykład kolumna `expected_num_interested_in_me` ma 78.5% braków — jeśli dla wierszy gdzie ta wartość istnieje match rate jest drastycznie inny, to może to być sygnał że ta cecha wpływa na wynik i nie należy jej po prostu imputować medianą. Yazan dzieli też dane na trzy zbiory (train/test/validation) a nie dwa jak ja — validation set jest kluczowy dla kalibracji i progowania, a ja robię oba te kroki na teście, co tak realnie na to patrząc jest wyciekami danych.

Yazan natomiast używa ADASYN zamiast SMOTE. ADASYN generuje więcej próbek syntetycznych dla trudniejszych przypadków mniejszości (tych blisko granicy klas), podczas gdy SMOTE generuje je równomiernie. W moim notatniku używałem SMOTE tylko do wizualizacji t-SNE, nie w modelach. ADASYN mógłby być lepszy dla moich danych bo klasy mocno się przenikają.

W pliku financial-distress-prediction kolumny z dużymi odchyleniami są normalizowane przez obcięcie do $\pm 2\sigma$:

```
def normalizer(x, df):
    upper = df[x].mean() + 2 * df[x].std()
    lower = df[x].mean() - 2 * df[x].std()
    df.loc[df[x] < lower, x] = lower
    df.loc[df[x] > upper, x] = upper
```

U mnie wiek ma zakres 18-55 przy średniej 26 — obcięcie do $\pm 2\sigma$ zredukowałoby ogon.

Optymalizacja hiperparametrów

Ja po prostu wpisuję hiperparametry ręcznie — `max_depth=5`, `n_estimators=300`, `max_iter=3000`. Nie robię GridSearch ani żadnej innej optymalizacji. Nie jestem w stanie stwierdzić jak bardzo każdy z nich był ważny.

W pliku yazana optymalizacja jest manualna ale znacznie bardziej systematyczna — dla każdego parametru robi sweep i rysuje krzywe PR + ROC:

```
for c in [0.0001, 0.001, 0.1, 1, 10, 25, 50, 100]:
    lsvm = svm.LinearSVC(C=c, dual=False)
    lsvm.fit(X_train_, y_train_)
    # rysuje PR i ROC dla każdego C
```

To pozwala zobaczyć jak zmiana parametru wpływa na krzywą a nie tylko na jedną metrykę. Nie jest to jednak w pełni optymalne podejście — m.in. nie robi CV, i wybiera parametry wizualnie a nie numerycznie.

Prawdopodobnie najlepszym podejściem byłoby nie rezygnowanie z GridSearch, i spojrzenie na atrybut `cv_results_` aby dowiedzieć się czegoś więcej o tym który hiperparametr okazał się najważniejszy:

```
from sklearn.model_selection import RandomizedSearchCV
search = RandomizedSearchCV(
    BalancedRandomForestClassifier(n_jobs=-1),
    {n_estimators: [100, 200, 300, 500],
     max_depth: [5, 10, 20, None],
     sampling_strategy: [all, 0.5]},
    scoring=average_precision, cv=5, n_iter=20
)
```

Jeszcze jeden problem — próg biznesowy dobieram empirycznie na zbiorze testowym. To jest wyciek danych. Powinienem być mieć validation set i tam szukać optymalnego progu, a na teście tylko ewaluować.

Uczenie modelu

W moim kodzie używam kilku modeli: DummyClassifier jako baseline, LogisticRegression, DecisionTree z `class_weight='balanced'`, BalancedRandomForest, EasyEnsemble, RandomForest z `class_weight`, i CalibratedClassifierCV do kalibracji. Mocną stroną jest że używam ensemble dedykowanych niezrównoważeniu (BalancedRF, EasyEnsemble) oraz kalibracji prawdopodobieństw — żaden z notebooków Kaggle tego nie robi.

Słabą stroną jest że wszystkie parametry są hardcoded. W notatnikach Kaggle testowane jest po kilka wariantów transformacji danych i konfiguracji modelu. Yazan testuje 3 rodziny modeli (SVM, RF, MLP) × 3 reżimy danych (unsampled, ADASYN, weighted) = 9 konfiguracji. Geo_Ir testuje 3 warianty LR (vanilla, SMOTE, balanced).

Żaden z notebooków (ani mój) nie używa XGBoost ani LightGBM. To jest standard na Kaggle dla danych tabelarycznych.

```
import xgboost as xgb
model = xgb.XGBClassifier(
    scale_pos_weight=len(y_train[y_train==0]) / len(y_train[y_train==1]),
    n_estimators=500, max_depth=6, learning_rate=0.1
)
```

`scale_pos_weight` robi to samo co `class_weight='balanced'` ale w kontekście gradient boosting. U mnie zamiast RF mógłbym użyć XGB z tym parametrem.

W pliku yazana używany jest też bardzo agresywny undersampling zamiast oversamplingu:

```
train0 = train_df[train_df[SeriousDlqin2yrs] == 0].sample(frac=0.06684)
train1 = train_df[train_df[SeriousDlqin2yrs] == 1].copy()
train_df = pd.concat([train0, train1], axis=0)
```

Prawdopodobnie należałoby się z tym bardziej pobawić — u mnie undersampling pojawia się tylko wewnątrz BalancedRF (per-drzewo), ale nie jako osobny krok preprocessing. Również ADASYN zamiast SMOTE w samych modelach (nie tylko wizualizacji) byłby warty przetestowania.

Ewaluacja

W moim notatniku ewaluacja jest dość bogata — `classification_report_imbalanced`, confusion matrix, krzywe ROC i PR z AUC/AP, Precision@K, MRR, Brier score, calibration curve, macierz kosztów i zysk

biznesowy. To jest więcej niż w notatnikach Kaggle gdzie geo_lr używa głównie FNR a yazan rysuje krzywe ale nie wypisuje wartości AUC.

Mimo to brakuje kilku rzeczy. Najważniejsza to walidacja krzyżowa — ani ja, ani notatniki Kaggle jej nie używają. Geo_lr pisze wynik na jednym zbiorze testowym, yazan też. U mnie z 610 próbek testowych (108 matchy), ± 5 próbek to $\pm 5\%$ recall. Bez CV nie wiem czy wyniki są stabilne.

Podczas mierzenia metryk nie robię cross-validate ręcznie — GridSearch robi to wewnętrznie ale nie pokazuje tego. Zgłoszenia Kaggle też tego nie robią. Prawidłowe podejście to:

```
from sklearn.model_selection import cross_validate, StratifiedKFold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_validate(clf, X, y, scoring=[roc_auc, average_precision],
                        cv=cv)
print(fAUC: {scores["test_roc_auc"].mean():.4f} ( $\pm$ 
      {scores["test_roc_auc"].std():.4f}))
```

Dla mojego przypadku powinienem był użyć GroupKFold z user_id jako grupami, żeby ten sam użytkownik nie był w treningu i teście w tym samym foldzie.

Kolejna rzecz — geo_lr i yazan pokazują znormalizowaną confusion matrix (po wierszach). U mnie jest tylko heatmap surowych liczb. Normalizacja lepiej pokazuje proporcje — od razu widać że 90% fraudów/matchy przeoczonych.

Podsumowanie

Największe braki w moim rozwiązaniu które widać po porównaniu z Kaggle:

1. **EDA nie obejmuje wszystkich cech** — sprawdziłem tylko 5 samoocen, a powinienem distploty wszystkich 47 cech.
2. **Brak analizy anomalii wielokolumnowych** — kombinacje wartości które nie mają sensu (np. expected_num_matches > 10 przy match rate = 0%).
3. **Brak normalizacji outlierów** — obcięcie do $\pm 2\sigma$ dla cech z długimi ogonami.
4. **Brak stacking** — każdy model testowany osobno, a kombinacja przez meta-model daje typowo +2-5% AP.

Mocne strony mojego rozwiązania których nie ma w notatnikach Kaggle:

- Ensemble dedykowane niezrównoważeniu (BalancedRF, EasyEnsemble)
- Kalibracja prawdopodobieństw (CalibratedClassifierCV, Brier score)
- Threshold tuning (F2 + analityczny p^* z macierzy kosztów)
- Macierz kosztów i zysk biznesowy
- Metryki rankingowe (Precision@K, MRR)

- Pipeline z ColumnTransformer (brak wycieku, reprodukowalność)
- Per-user split (realistyczny — symuluje nowych użytkowników)